CMPUT 331, Assignment 7: Language

Before beginning work on this assignment, carefully read the Assignment Submission Specifications posted on Canvas.

Introduction

Language identification plays a crucial role in cryptanalysis, enabling the recognition of a text's origin based on statistical and structural properties. Different languages exhibit distinct frequency distributions of letters and symbols, which can be analyzed through techniques including sorted symbol distribution (SSD), decomposition patterns, and more. By comparing these distributions against known language profiles, one can infer the most likely source language of a given ciphertext.

Watch the video "Lecture on Language Decoding" which can also be found under the "Decoding Human Language" module on Canvas (you may find 18:50-24:19 of the video most helpful). One of the problems considered in the video was that of *ciphertext language identification* (CLI), the task of determining the language of the plaintext. Three methods for solving this problem were presented:

1. Return the language with the closest sorted symbol distribution.
2. Return the language with the closest decomposition pattern distribution.
3. Return the language which gives the best trial decipherment.

In this assignment, you will implement simplified versions of *the first two* of these methods, and test them on a set of ciphers in ten languages: English, French, Italian, Spanish, German, Dutch, Polish, Russian, Bulgarian, and Greek. Since some of the files you use in this assignment will necessarily contain unicode symbols, please remember to open the files using `encoding="utf8"` and to convert all text into uppercase using **.upper()**for consistency in processing. In addition, in this assignment you can use the python function **isalpha()** to determine whether a character is a valid letter for your calculations.

You will produce three files for this assignment:

- **a7.py**: Python file to problems 1, 2, 3 and 4.
- **a7.txt**: Written responses to problem 4.
- a README file (**README.txt**)

Submit your files in a zip file named **331-as-7.zip**.

Problem 1

Given a text, its *sorted symbol distribution* (SSD) is a frequency distribution over the set of symbol frequency ranks. The most frequent symbol in the text has rank 1, and its relative frequency is the probability of rank 1 in the SSD. In general, $P_{SSD}(i)$ is the relative frequency of the ith most frequent symbol in the text, where i ranges from 1 to the size of the alphabet (excluding whitespace and punctuation). For example, in a typical English text, $P_{SSD}(3) = 0.0817$ (using the textbook's relative letter frequencies), because "A" is the third most frequent letter, and its relative frequency is 0.0817.

The key insight here is that SSD is invariant under monoalphabetic substitution. A monoalphabetic substitution may change what the ith most frequent symbol is, but it will not change its relative frequency, and so $P_{SSD}(i)$ will not change. As different languages have different SSDs, we can use this to identify the language of a ciphertext. First, compute the SSDs for various languages from sample texts. Second, compute the SSDs of the ciphertext(s). Third, determine which language has the closest SSD to the SSD of the ciphertext; return that language.

For the purposes of this assignment, you will compare frequency distributions using the sum-of-squared-differences method. So, given two SSDs P_1 and P_2, the distance between them is:

$$\sum_{i=1}^{|alphabet|} \left(P_1(i) - P_2(i)\right)^2$$

It may be that the two texts have a different number of symbol types. For example, a short English ciphertext may contain only twenty-four symbols, but a longer one may contain all twenty-six letters. In such cases, you should take the larger of two alphabet sizes, and assign a probability of zero to unseen ranks. (In other words, undefined probabilities are zeros.)

Write a function `cliSSD(ciphertext, files)` in the a7.py module, taking two arguments: `ciphertext`, a string containing the ciphertext, and `files`, a list of strings, with each string being a path to a file containing a sample text in a single language. It should return a dictionary, where each key is a string corresponding to an item in `files`. The value of each key is a float distance between the SSD of the text in the file, and the SSD of the ciphertext.

Problem 2

Given a word, its *decomposition pattern* is a k-ary tuple, where k is the number of distinct letter types in the word. Each element of the tuple is a positive integer which corresponds to one type of letter, and its value is the number of times that letter appears in the word. The elements of the tuple are in numerically sorted order, from highest to lowest. It is implicit in this specification that the sum of the elements of the decomposition patterns should be equal to the number of letters in the word. Here are some examples:

- "SEEMS" $\rightarrow (2, 2, 1)$.
- "BEAMS" $\rightarrow (1, 1, 1, 1, 1)$.
- "WERE" $\rightarrow (2, 1, 1)$.

Decomposition patterns are resistant to monoalphabetic encipherment: a word with five distinct symbols, such as "BEAMS" will always encipher to a word with five distinct symbols.

Given a text, its *decomposition pattern distribution* (DPD) is the probability distribution over the decomposition patterns of the words in the text. For example, if 0.19% of all word tokens in a text have pattern (2,2,1), then $P_{DPD}((2, 2, 1)) = 0.0019$. We can use DPDs to perform CLI just as we did with SSDs: compute the DPD of the ciphertext and some samples of various languages, and decide which of the samples has the closest DPD to that of the ciphertext. Again, use sum-of-squared-differences to compare DPDs.

It is often the case that the decomposition pattern in the ciphertext does not match exactly with that of a sample text. In such scenarios, it is important to include all patterns from both the ciphertext and the sample in the comparison and assign a probability of zero to the patterns not observed for each side. (In other words, undefined probabilities are zeros.)

Write a function `cliDPD(ciphertext, files)` in the a7.py module, taking two arguments: `ciphertext`, a string containing the ciphertext, and `files`, a list of strings, with each string being a path to a file containing a sample text in a single language. It should return a dictionary, where each key is a string corresponding to an item in `files`. The value of each key is a float distance between the DPD of the text in the file, and the DPD of the ciphertext.

Important Note:

1. To get the words of a text, split it according to whitespace using `text.split()`, then remove every non-letter character (any character for which `isalpha()` is `False`) from each token. A token with no letters left, such as "-" or "1990", still counts as a word: it is an empty word with a decomposition pattern of `()`, and it is included in the DPD like any other word. For example, the text `"The cat - 1990."` gives the words `THE`, `CAT`, and two empty words, with patterns $(1, 1, 1)$, $(1, 1, 1)$, (), (). So its DPD is $\{(1, 1, 1) : 0.5,\ () : 0.5\}$.

Problem 3

Included with this assignment are 50 ciphers, 5 each from 10 different languages. The filenames reveal the correct ciphertext languages. For example, the file "ciphertext_en_1.txt" contains an English ciphertext. Apply the two methods presented to these 50 ciphers. You can evaluate them based on the accuracy: determine how many ciphers, out of 50, the method can correctly identify the ciphertext language.
First, write a function `cliSSDTest(ciphertext_files, sampletext_files)` in the a7.py module to implement the SSD method.

Second, write a function `cliDPDTest(ciphertext_files, sampletext_files)` in the a7.py module to implement the DPD method.

Both functions should take two arguments: `ciphertext_files`, a list of strings, with each string being a path to a file containing a ciphertext in a single language (e.g., "ciphertext_en_1.txt"), and `sampletext_files`, a list of strings, with each string being a path to a file containing a sample text in a single language (e.g., "sample_en.txt"). It should return a dictionary, where each key is a string corresponding to an item in `ciphertext_files`, and the value of each key is a string corresponding to an item in `sampletext_files`. The value should represent the sample file that has the closest distance to the key which is the ciphertext file, indicating the identified ciphertext language.

An example output is as follows.

```
{"ciphertext_el_1.txt": "sample_el.txt",
 "ciphertext_el_2.txt": "sample_el.txt",
 ...
 "ciphertext_nl_5.txt": "sample_nl.txt",
 "ciphertext_nl_6.txt": "sample_nl.txt"}
```

Please keep solutions to problem 3 within a runtime of 1 second per cipher text file.

Problem 4

Write a function `generateMapping(dictionary)` in the a7.py module, taking an argument: `dictionary`, the resulting dictionary from each of the functions in Problem 3. It should return a classification mapping (dictionary) using a structured key-value format. Each of your entry should follow the pattern:

```
(actual_language, predicted_language): count
```

where "actual_language" is the language code of the true language of the cipher, "predicted_language" is language code of the language it was identified as, and "count" is the number of instances this classification occurred. **Please refer to the table at the bottom of this page for the language codes.**

For example, if six English ciphers were correctly classified as English, two were misclassified as German, and one as Russian, the output would include:

```
{
    ("en", "en"): 6,
    ("en", "de"): 2,
    ("en", "ru"): 1
}
```

You can then use the provided function `generateMatrix()` to generate a visualization of the confusion matrix based on the classification mapping of your two methods. The true language should be on the vertical axis, and the identified language on the horizontal axis.
Note: Do not call `generateMatrix()` from within `generateMapping()`. You can use the `test()` function.

Examine the two matrices for any noticeable patterns and answer the following question: **Which language is the most confused?** Provide your response in "a7.txt". Replace the blanks from the template provided with your answers.

Language Code Conversion Table

Code	Language
bg	Bulgarian
de	German
el	Greek
en	English
es	Spanish
fr	French
it	Italian
nl	Dutch
pl	Polish
ru	Russian